

Mining — Templates, CPU Miner & Stratum

How the node produces work for miners, how a miner solves proof-of-work, and the Stratum protocol that connects real mining hardware to the node.

SUBJECT hathor-core · Part II · the node, end to end

CHAPTER 37 · Part II · The Node

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Mining · Block template · Proof-of-work · CPU miner · Thread pool · Stratum · Jobs & shares · Difficulty · Merged mining · Aux-PoW

Table of Contents

Chapter 37 — Mining: Templates, CPU Miner & Stratum	3
37.1 Localization	3
37.2 What it does and why it exists	4
37.3 The concepts it rests on	5
37.4 The code, walked	6
37.4.1 The block template — what goes in, and why	6
37.4.2 How the node builds a template	8
37.4.3 The CPU mining service — solving the puzzle	9
37.4.4 Stratum — the standard miner-to-node protocol	10
37.4.5 Merged mining — Hathor as a free side-effect of Bitcoin	15
37.4.6 The solved block, back into the node	16
37.4.7 The job/share lifecycle, end to end	17
37.5 Why Stratum, and not naive polling	17
37.6 How it plugs into the lifecycle	18
Recap	19

Chapter 37 — Mining: Templates, CPU Miner & Stratum

What you'll learn in this chapter

- What a **block template** is — the well-formed candidate block the node hands a miner — and exactly what goes into one and why (parents, reward, weight).
- How the node's **CPU mining service** solves proof-of-work by grinding a nonce, and why that grinding must stay off the reactor.
- The **Stratum protocol**: the standard, long-lived connection between mining hardware and a node, built on a **job → share** model. This is Stratum's canonical treatment in the book.
- **Merged mining**: mining Hathor as a side-effect of mining Bitcoin, using an auxiliary proof-of-work.
- How a solved block re-enters the node — `submit_block` → the vertex handler (Ch. 33) → consensus (Ch. 32) — closing the loop from "found a block" to "ledger changed."

Mining is where new coins are minted and where the chain's security comes from. We have already met the theory — what proof-of-work is, what weight measures, why "most work wins" (Chapters 6 and 9). This chapter is the plumbing: the three packages that turn that theory into a running service. `hathor/mining/` builds the candidate block and can solve it on a CPU. `hathor/stratum/` is the network protocol that lets a real miner — a separate program, possibly on dedicated hardware — talk to the node. `hathor/merged_mining/` lets a Bitcoin miner mine Hathor for free on the side.

We start with the concept that everything here orbits: the template.

37.1 Localization

Three sibling packages, plus a few CLI entry points that drive them:

```

hathor-core/
├── hathor/
│   ├── mining/
│   │   ├── __init__.py           ← exports BlockTemplate, BlockTemplates
│   │   ├── block_template.py     ← BlockTemplate (NamedTuple) + BlockTemplates (list)
│   │   ├── cpu_mining_service.py ← CpuMiningService: grinds the nonce on a CPU
│   │   └── ws.py                 ← websocket helper for mining (not covered here)
│   ├── stratum/
│   │   ├── __init__.py
│   │   ├── stratum.py           ← JSONRPC, StratumProtocol, StratumFactory, StratumClient
│   │   └── resources.py         ← HTTP resource exposing miner statistics
│   └── merged_mining/
│       ├── coordinator.py       ← MergedMiningCoordinator, MAGIC_NUMBER, aux-pow build
│       ├── bitcoin.py           ← Bitcoin block/merkle primitives
│       └── bitcoin_rpc.py       ← talks to a bitcoind over JSON-RPC
└── hathor_cli/
    ├── mining.py                ← `hathor-cli mining`: standalone HTTP-polling CPU miner
    ├── stratum_mining.py        ← `hathor-cli stratum_mining`: standalone Stratum CPU mine
    └── merged_mining.py         ← `hathor-cli merged_mining`: the merge-mining coordinator

```

The node itself does not normally mine. In production a node serves work (via Stratum) and accepts solutions; the actual nonce-grinding happens in a separate miner process — often the standalone CLI tools above, or third-party mining software. The `CpuMiningService` inside the node exists mostly for tests, the simulator, and the standalone `hathor-cli mining` tool. Keep that split in mind: **the node hands out work and validates results; miners do the work.**

Context. Mining sits at the edge of the node's monetary and security responsibilities (Ch. 0 §0.2, jobs 3 and 5). It produces the blocks that mint HTR and that consensus (Ch. 32) uses to order the DAG. A found block is not special once it arrives — it flows through the same ingestion pipeline (Ch. 33) as any vertex received from a peer. Mining is therefore best understood as a producer feeding the front door we already studied.

37.2 What it does and why it exists

A miner needs two things from the node, and the node needs one thing back.

The miner needs, first, a **candidate block** to work on — a block that is already valid in every respect except that its proof-of-work is not yet solved. It must already point at the right parents, already pay the correct reward, already carry the right difficulty. The

only blank left for the miner to fill is the **nonce**¹ (and, optionally, the timestamp). If the node handed out a half-formed block, every miner would have to re-derive the chain's current state for itself — defeating the point of a full node.

The miner needs, second, to be told how hard the puzzle is — the **weight** (Hathor's difficulty unit; recap below). Without a target, "solve the proof-of-work" has no finish line.

The node needs, in return, to **accept a solved block** and fold it into the ledger: check the proof-of-work is genuine, check it still extends the current best tip, then push it through verification and consensus.

The package that bundles the first two — the candidate block plus its difficulty plus the surrounding constraints — into one object is the **block template**. Producing templates, solving them, and accepting solutions is the whole job of this chapter.

Recap — mining secures the chain and mints coins (full treatment in Ch. 6 §6.6 and Ch. 9). A blockchain stays honest because rewriting history means redoing proof-of-work, which costs real electricity (Ch. 6). Each block carries a **weight** — a number measuring how much work it represents — and consensus prefers the history with the most accumulated weight (Ch. 9, Ch. 32). Mining is simultaneously how that work gets done and how new HTR enters circulation, as the block's reward output. → full treatment in Ch. 6 and Ch. 9.

37.3 The concepts it rests on

Three ideas from earlier chapters carry the weight here. We recap each briefly; none is re-taught from scratch.

Recap — proof-of-work, weight, nonce, target (full treatment in Ch. 9). Every Hathor vertex has a **weight**: a float that is the base-2 logarithm of the expected number of hash attempts needed to solve it. A weight of w means roughly 2^w tries. From the weight the code derives a numeric **target**: a solution is valid when the block's hash, read as a big integer, is **below** the target. The miner varies one field — the **nonce**, a counter — recomputes the hash, and checks. Larger weight $\Rightarrow$ smaller target $\Rightarrow$ more tries. → full treatment in Ch. 9.

Recap — never block the reactor; use a thread or process pool (full treatment in Ch. 2 and Ch. 16). The node runs on a single-threaded event loop, the Twisted reactor. Any function that runs for a long time without returning freezes the whole node — no peers served, no timers fired. Grinding billions of hashes is exactly such a long-running CPU job. So CPU mining never happens inside the node's reactor: it runs in a separate process (the standalone CLI miners) or is offloaded to a worker. The reactor only handles the fast, I/O-bound parts: sending a job, receiving a share. → full treatment in Ch. 2 and Ch. 16.

Recap — a block is a vertex (full treatment in Ch. 8 and Ch. 25). A `Block` is one of Hathor's two vertex types. Like any vertex it has parents (graph edges) and a weight; unlike a transaction it has no inputs, and its single output mints the reward. A Hathor block points at three parents: the previous block plus, usually, two transactions it confirms. → full treatment in Ch. 8 and Ch. 25.

With those in hand, the code becomes readable.

37.4 The code, walked

We build the picture in the order data flows: first the template (what work looks like), then solving it (the CPU service), then the protocol that distributes work to real miners (Stratum), then the merged-mining variant, and finally the path a solved block takes back into the node.

37.4.1 The block template — what goes in, and why

`BlockTemplate` is a `NamedTuple`² — an immutable record with named fields — defined at `hathor/mining/block_template.py:29`. Read its fields as a contract: "here is everything you need to build a mineable block, and nothing you need to ask the chain for again."

```
class BlockTemplate(NamedTuple):
    versions: set[int]
    reward: int          # reward unit value, 64.00 HTR is 6400
    weight: float         # calculated from the DAA
    timestamp_now: int    # the reference timestamp the template was generated for
    timestamp_min: int    # min valid timestamp
    timestamp_max: int    # max valid timestamp
    parents: list[bytes]  # required parents: always a block, at most 2 txs
    parents_any: list[bytes] # extra parents to choose from when there are options
    height: int           # metadata
    score: int            # metadata
    signal_bits: int      # signal bits for blocks generated from this template
```

`hathor/mining/block_template.py:29`

Each field models something concrete:

- **reward** is how many HTR the new block mints, in the smallest unit (the comment notes `64.00 HTR is 6400` — two implied decimals). It is fixed by the block's height, because Hathor's issuance schedule (halvings) is a function of height. The miner cannot choose it; a wrong reward would make the block invalid.
- **weight** is the difficulty, computed by the **DAA**³ (difficulty-adjustment algorithm, Ch. 9). It sets the finish line. The template fixes a minimum weight so a miner cannot hand back a block that is too easy to be worth anything.
- **parents** and **parents_any** together encode the block's graph edges. A Hathor block always has exactly three parents. The first is mandatory — the previous block (the tip being extended). The remaining slots are filled with transactions the block will confirm. **parents** holds the ones that must be included; **parents_any** is a menu the miner picks the rest from when there is more than one valid choice. (Why split them? So two miners building on the same tip can still produce different blocks by choosing different transactions to confirm — useful diversity.)
- **timestamp_min** / **timestamp_max** / **timestamp_now** bound the block's timestamp. A block cannot be older than its parents or too far in the future; the template states the legal window so the miner's clock cannot drift out of bounds.
- **signal_bits** carries the miner's feature-activation vote (Ch. 38). The template pre-fills the bits the node wants signalled.
- **height** and **score** are metadata: the new block's height (parent height + 1) and its accumulated score (parent score + this block's work). The miner does not need them to mine, but the node uses them.

The template knows how to turn itself into an actual block. `generate_mining_block` (`block_template.py:53`) fills the blanks — picks random parents from the menu, attaches the reward output paying the miner's address, sets the weight and timestamp — and returns a `Block` ready to grind:

```
def generate_mining_block(self, rng, address=None, timestamp=None, ...):
    parents = list(self.get_random_parents(rng))
    block_timestamp = min(max(base_timestamp, self.timestamp_min), self.timestamp_max)
    tx_outputs = []
    if self.reward:
        output_script = create_output_script(address) if address is not None else b''
        tx_outputs = [TxOutput(self.reward, output_script)]
    block = cls(outputs=tx_outputs, parents=parents, timestamp=block_timestamp,
               weight=self.weight, signal_bits=self.signal_bits, ...)
    return block
```

`hathor/mining/block_template.py:53`

Note `get_random_parents` (`block_template.py:87`): it always returns exactly three parents — the required ones plus a random sample from `parents_any` to fill the gap — and asserts that length. The "three parents" invariant of a Hathor block is enforced right here.

`BlockTemplates` (`block_template.py:130`) is a thin `list` subclass holding one or more templates (one per competing best tip) plus the storage handle.

`choose_random_template` (`:136`) picks one. In normal operation there is a single best tip, so the list has one entry.

37.4.2 How the node builds a template

The node-side entry point is `HathorManager.make_block_template` (`hathor/manager.py:666`). It loads the parent block, gathers candidate parent transactions, then delegates to `_make_block_template` (`manager.py:696`), which is where the interesting decisions live.

The reward comes from the difficulty algorithm by height:

```
height = parent_block.get_height() + 1
...
reward=self.daa.get_tokens_issued_per_block(height),
```

`hathor/manager.py:742, :757`

The weight is the larger of "what the DAA says the next block should be" and a computed floor that guarantees the block actually moves the chain's score forward:

```
min_significant_weight = calculate_min_significant_weight(
    parent_block_metadata.score, 2 * self._settings.WEIGHT_TOL)
weight = max(
    self.daa.calculate_next_weight(parent_block, timestamp, self.tx_storage.get_parent_block),
    min_significant_weight,
)
```

`hathor/manager.py:734`

`calculate_next_weight` is the DAA from Chapter 9 — it tunes difficulty so blocks arrive at a steady rate. The `min_significant_weight` floor exists because a block whose weight is too small would not raise the accumulated score enough to count; the node refuses to hand out such trivial work.

The parents are assembled from the parent block plus the gathered transactions, then packed so the template carries either three fixed parents or a fixed set plus a menu:


```

parents = [parent_block.hash] + list(parent_txs.must_include)
parents_any = parent_txs.can_include
if len(parents) + len(parents_any) == 3:
    parents.extend(sorted(parents_any))
    parents_any = []
assert 1 <= len(parents) <= 3, 'Impossible number of parents'

```

hathor/manager.py:743

Finally the template is assembled with both block versions a miner may use (a regular block or a merge-mined block) and the feature-signal bits:

```

return BlockTemplate(
    versions={TxVersion.REGULAR_BLOCK.value, TxVersion.MERGE_MINED_BLOCK.value},
    reward=self.daa.get_tokens_issued_per_block(height),
    weight=weight,
    ...
    signal_bits=self._bit_signaling_service.generate_signal_bits(block=parent_block),
)

```

hathor/manager.py:755

`get_block_templates` (`manager.py:650`) wraps this and is the method mining APIs and the Stratum server actually call. Its docstring notes it is a "cached version... cache is invalidated when latest_timestamp changes" — templates are reused until the chain state moves, so the node is not rebuilding a template for every miner poll.

37.4.3 The CPU mining service — solving the puzzle

Once you have a block, mining it is a loop: set a nonce, hash, check against the target, increment, repeat. That loop lives in `CpuMiningService` (`hathor/mining/cpu_mining_service.py:25`). The heart is `start_mining` :

```

def start_mining(vertex, *, start=0, end=MAX_NONCE, ..., should_stop=lambda: False):
    pow_part1 = vertex.calculate_hash1()
    target = vertex.get_target()
    vertex.nonce = start
    while vertex.nonce < end:
        ...
        result = vertex.calculate_hash2(pow_part1.copy())
        if int(result.hex(), vertex.HEX_BASE) < target:
            return result
        vertex.nonce += 1
    return None

```

hathor/mining/cpu_mining_service.py:50

Three things to notice. The hash is split into two halves — `calculate_hash1` computes the part that does not depend on the nonce, once, and `calculate_hash2` finishes the part that does, every iteration. That is a real optimization: the nonce-independent prefix is hashed a single time and copied (`pow_part1.copy()`) per attempt. The win condition is the target comparison from Chapter 9: hash-as-integer below target. And

`MAX_NONCE = 2**32 ( :22 )` bounds the search; if the loop exhausts the nonce space without a hit, it bumps the timestamp (`update_time` , every 2 seconds) and restarts, because changing the timestamp changes the hash and reopens the search.

`resolve ( :26 )` wraps `start_mining` , stores the found hash on the vertex, and handles the special case of a token-creation transaction whose token id is its own hash.

This service is injected into the manager (`manager.py:112` , stored at `:199`) so collaborators have a single mining implementation to call. Block mining in production is external — real miners do that work over Stratum. But the in-node service is not idle: it solves the small proof-of-work on **user transactions**. Every Hathor transaction carries a (small, 4-byte-nonce) proof-of-work as anti-spam, so when a user submits a transaction through the wallet's REST API the node grinds that nonce for them — for example in `send_tokens ( hathor/wallet/resources/send_tokens.py:134 )` and the thin-wallet endpoint (`hathor/wallet/resources/thin_wallet/send_tokens.py:267`). That grind is offloaded to a dedicated thread pool so it never blocks the reactor:

```
self.pow_thread_pool = ThreadPool(minthreads=0, maxthreads=settings.MAX_POW_THREADS,
                                  name='Pow thread pool')
```

`hathor/manager.py:234` (started at `:310`); the thin-wallet endpoint dispatches into it via `deferToThreadPool` at `wallet/resources/thin_wallet/send_tokens.py:182`

The same service also mines the genesis vertices at startup (`hathor/transaction/genesis.py:71`). For block mining specifically, where you see the service is the standalone CLI miner. `hathor_cli/mining.py` is a small program that polls the node's HTTP mining endpoint, mines whatever block it gets back, and POSTs the solution:

```
def worker(q_in, q_out):
    from hathor.mining.cpu_mining_service import CpuMiningService
    block, start, end, sleep_seconds = q_in.get()
    CpuMiningService().start_mining(block, start=start, end=end, sleep_seconds=sleep_seconds)
    q_out.put(block)
```

`hathor_cli/mining.py:35`

It runs the grind in a separate `Process` (`mining.py:125`), never inside a reactor — exactly the "don't block the loop" rule from the recap. This HTTP-polling style is the naive approach we contrast against Stratum in §37.5.

37.4.4 Stratum — the standard miner-to-node protocol

The CLI HTTP miner works, but it is wasteful: it asks for a block, mines it, asks again. Between requests there is no live connection; if the chain tip moves, the miner keeps grinding stale work until its next poll. Real mining wants a persistent connection over which the node pushes fresh work the instant it changes, and the miner streams back partial progress. That protocol is **Stratum**.

A generic mining pool, first

Before the Hathor code, hold the generic picture — it is identical across every cryptocurrency that uses Stratum (Bitcoin included).

Imagine a pool operator coordinating many miners. The operator keeps an open TCP connection to each miner. The exchange is:

1. The miner **subscribes** — "I'm here, here's the address to pay me."
2. The operator sends a **job**: "here is the work; here is its difficulty."
3. The miner grinds. Each time it finds a hash that beats the job's (deliberately easy) difficulty, it submits a **share** — proof it is working.
4. The operator checks the share. Most shares are not full blocks; they only prove effort. But occasionally a share also beats the network's (much harder) difficulty — that share **is a block**, and the operator broadcasts it.
5. Whenever the work goes stale (a new block appears on the network), the operator pushes a **fresh job**, and the miner abandons the old one.

Two difficulties, then: the easy **share difficulty** (tuned so each miner submits a share every few seconds, giving a steady signal of its hash-rate) and the hard **network difficulty** (the real bar for a block). This job → share model is the entire conceptual core. Everything in `stratum.py` is this picture in code.

Stratum in Hathor: the layers

The protocol is JSON-RPC 2.0⁴ over a line-delimited TCP connection. The class stack reflects that:

- **JSONRPC** (`hathor/stratum/stratum.py:181`) — a Twisted `LineReceiver` that parses each newline-terminated line as a JSON-RPC message and dispatches it to `handle_request`, `handle_result`, or `handle_error`. It is the shared transport for both server and client.
- **StratumProtocol** (`stratum.py:337`) — the server side, one instance per connected miner. It holds that miner's jobs, address, and statistics. This is where subscribe and submit are handled.
- **StratumFactory** (`stratum.py:733`) — the Twisted factory⁵ that builds a `StratumProtocol` per connection and holds shared state (all miner protocols, the transaction-mining queue). It also subscribes to the node's events so it knows when to push fresh jobs.
- **StratumClient** (`stratum.py:813`) — the miner side, used by `hathor-cli` `stratum_mining`. It spawns worker processes to grind and submits shares.

`StratumProtocol.handle_request` (`stratum.py:404`) is the dispatcher. It accepts exactly two methods — `subscribe` and `submit` (with `mining.` prefixes accepted too) — and rejects anything else. If the node is still syncing it answers `NODE_SYNCING` rather than hand out work on an incomplete chain (`:418`).

Subscribe → first job

When a miner subscribes, `handle_subscribe` (`stratum.py:441`) decodes the payout address, records whether the miner wants to mine transactions or do merged mining, registers the protocol in the factory's `miner_protocols`, answers `ok`, and immediately calls `job_request` to send the first job:

```
self.factory.miner_protocols[self.miner_id] = self
self.send_result('ok', msgid)
self.subscribed = True
self.job_request()
```

`hathor/stratum/stratum.py:472`

`create_job` (`stratum.py:619`) builds a `ServerJob`: it generates the work (a block, or a queued transaction), assigns a UUID, computes the share weight for this miner, and sets the job's weight to the smaller of the share weight and the real work's weight:

```
share_weight = self.calculate_share_weight()
job.weight = min(share_weight, tx.weight)
```

`hathor/stratum/stratum.py:639`

This is the two-difficulties idea in code. `calculate_share_weight` (`stratum.py:693`) looks at how fast the miner has been solving recent jobs and picks a weight aimed at one share every `AVERAGE_JOB_TIME` (5 seconds, `:348`). A fast miner gets harder shares; a slow one gets easier shares — so every miner reports in at a steady cadence regardless of hardware. The job also gets a timeout (`:648`): if the miner does not submit within the window, the server pushes a fresh job rather than letting it grind stale work.

The actual work is built by `create_job_tx` (`stratum.py:656`). Blocks are the default; transactions are prioritized only if the miner opted into `mine_txs`:

```
block = self.manager.generate_mining_block(
    data=data, address=self.miner_address, merge_mined=self.merged_mining)
```

`hathor/stratum/stratum.py:681`

That call lands in `HathorManager.generate_mining_block` (`manager.py:769`), which uses the template machinery from §37.4.2. The loop is closed: subscribe → server asks the manager for a template → fills it into a block → ships it as a job.

The job sent to the miner carries only what it needs: the mining header without the nonce, the job id, the nonce size, and the (share) weight:

```

job_data = {
    'data': job.tx.get_mining_header_without_nonce().hex(),
    'job_id': job.id.hex,
    'nonce_size': job.tx.SERIALIZATION_NONCE_SIZE,
    'weight': float(job.weight),
}

```

hathor/stratum/stratum.py:608

`SERIALIZATION_NONCE_SIZE` differs by vertex type — a block's nonce is 16 bytes (`hathor/transaction/block.py:46`), a transaction's is 4 (`hathor/transaction/transaction.py:58`). The 4-byte tx nonce is why mining a transaction uses a short 1-second job timeout (`TX_MAXIMUM_JOB_TIME` , `stratum.py:351`): a 4-byte search space is small enough to exhaust quickly, so the timestamp must be refreshed often.

Submit → checking the share, finding a block

When the miner finds a hash that beats the job weight, it submits the nonce.

`handle_submit` (`stratum.py:479`) is the most consequential method in the file. Walk it:

1. **Validate the request** — `job_id` and `nonce` present, job id a real UUID (: 492).
2. **Find the job** — look it up; reject `JOB_NOT_FOUND` if unknown (:504).
3. **Reject stale shares** — if this is not the current job or it was already submitted, answer `STALE_JOB` (:512). This guards against a miner submitting work for a job the chain has already moved past.
4. **Reconstruct and re-hash** the block with the submitted nonce (:518 – :528).
5. **Verify the share** — re-run proof-of-work against the job's (easy) weight:

```

try:
    verifier.verify_pow(tx, override_weight=job.weight)
except PowError:
    self.log.error('bad share, discard', ...)
    return self.send_error(INVALID_SOLUTION, msgid, ...)

```

hathor/stratum/stratum.py:539

A failing share means the miner lied about doing the work — discard it. A passing share proves effort. The server marks the job submitted, answers `ok`, and **immediately schedules the next job** so the miner never idles:

```

self.send_result('ok', msgid)
self.manager.reactor.callLater(0, self.job_request)

```

hathor/stratum/stratum.py:552

1. **Is the share also a block?** Re-run proof-of-work, this time against the real network weight:

```

try:
    verifier.verify_pow(tx)          # no override: real difficulty
except PowError:
    self.log.info('high hash, keep mining', tx=tx)
    return
else:
    self.log.info('low hash, new block candidate', tx=tx)

```

hathor/stratum/stratum.py:555

Most shares fail this second check — they proved effort but are not blocks. The miner keeps going. But when a share passes, it is a genuine block, and the server submits it to the node:

```

if isinstance(tx, Block):
    try:
        self.manager.submit_block(tx)
        self.blocks_found += 1
    except (InvalidNewTransaction, TxValidationError) as e:
        self.log.warn('block propagation failed', block=tx, error=e)

```

hathor/stratum/stratum.py:564

That `submit_block` call is the bridge back into the node, covered in §37.4.6.

Pushing fresh work when the tip moves

The last piece is staying current. When a new block is accepted anywhere — found locally or arriving from a peer — every miner's current job becomes stale.

`StratumFactory.start` (`stratum.py:773`) subscribes to the node's pub-sub bus (Ch. 30) for exactly this:

```

def start(self):
    def on_new_block(event, args):
        tx = args.__dict__['tx']
        if isinstance(tx, Block):
            self.update_jobs()
    self.manager.pubsub.subscribe(HathorEvents.NETWORK_NEW_TX_ACCEPTED, on_new_block)

```

hathor/stratum/stratum.py:773

`update_jobs` (`stratum.py:765`) calls `job_request` on every subscribed miner, pushing each a fresh template built on the new tip. This event-driven push — rather than the miner polling — is the efficiency that Stratum buys, and it is why the factory must be started (subscribed) during the node's boot.

37.4.5 Merged mining — Hathor as a free side-effect of Bitcoin

Merged mining lets a miner secure two chains with one set of hashes. The idea: Bitcoin's proof-of-work hashes a Bitcoin block header; if you commit a Hathor block's hash inside that Bitcoin block (in its coinbase transaction), then a Bitcoin solution that happens to also clear Hathor's difficulty is simultaneously a valid Hathor block. Hathor rides Bitcoin's hash power for free.

The proof that "this Bitcoin work also solved this Hathor block" is an **auxiliary proof-of-work** (aux-pow). In Hathor it is `BitcoinAuxPow` (`hathor/transaction/aux_pow.py:24`), a record of the Bitcoin header pieces and the **Merkle path**⁶ needed to show the Hathor block's hash was committed in Bitcoin's coinbase:

```
class BitcoinAuxPow(NamedTuple):
    header_head: bytes      # 36 bytes
    coinbase_head: bytes    # variable
    coinbase_tail: bytes    # variable
    merkle_path: list[bytes] # each element 32 bytes
    header_tail: bytes      # 12 bytes
```

`hathor/transaction/aux_pow.py:24`

`calculate_hash` (`aux_pow.py:38`) reconstructs the Bitcoin header hash from those pieces; that hash is the merge-mined block's hash. `verify` (`:46`) checks two things: that Hathor's `MAGIC_NUMBER` (`b'Hath'`, `hathor/merged_mining/coordinator.py:55`) appears in the coinbase at the expected place — proving the commitment is to Hathor and not slipped in elsewhere — and that the Merkle path is not abusively long.

The coordinator that orchestrates this is `MergedMiningCoordinator` (`hathor/merged_mining/coordinator.py:1124`), run as the standalone `hathor-cli merged_mining` service. It is an asyncio server (not Twisted — this is a self-contained tool) that talks to a `bitcoind` over JSON-RPC and to a Hathor node, builds a combined job, and on a winning share constructs the aux-pow and forwards it to Hathor:

```
def build_aux_pow(self, work):
    bitcoin_header, coinbase_tx = self._make_bitcoin_block_and_coinbase(work)
    ...
    coinbase_head, coinbase_tail = coinbase.split(block_base_hash)
    return BitcoinAuxPow(header_head, coinbase_head, coinbase_tail,
                          list(self.merkle_path), header_tail)
```

`hathor/merged_mining/coordinator.py:230`

On the node side, the merged path rejoins the ordinary one: a `MergeMinedBlock` carrying an `aux_pow` is submitted through the same Stratum `handle_submit` (the `aux_pow` branch at `stratum.py:522`) and the same `submit_block`. Merged mining is a different way to produce the proof-of-work, not a different way to accept a block.

37.4.6 The solved block, back into the node

Whatever produced it — Stratum, the CLI miner, merged mining — a found block re-enters through `HathorManager.submit_block` (`hathor/manager.py:796`). It does two checks, then hands off:

```
def submit_block(self, blk):
    parent_hash = blk.get_block_parent_hash()
    best_block_hash = self.tx_storage.get_best_block_hash()
    if parent_hash != best_block_hash:
        self.log.warn('submit_block(): Ignoring block: parent not a tip', ...)
        return False
    ...
    return self.propagate_tx(blk)
```

`hathor/manager.py:796`

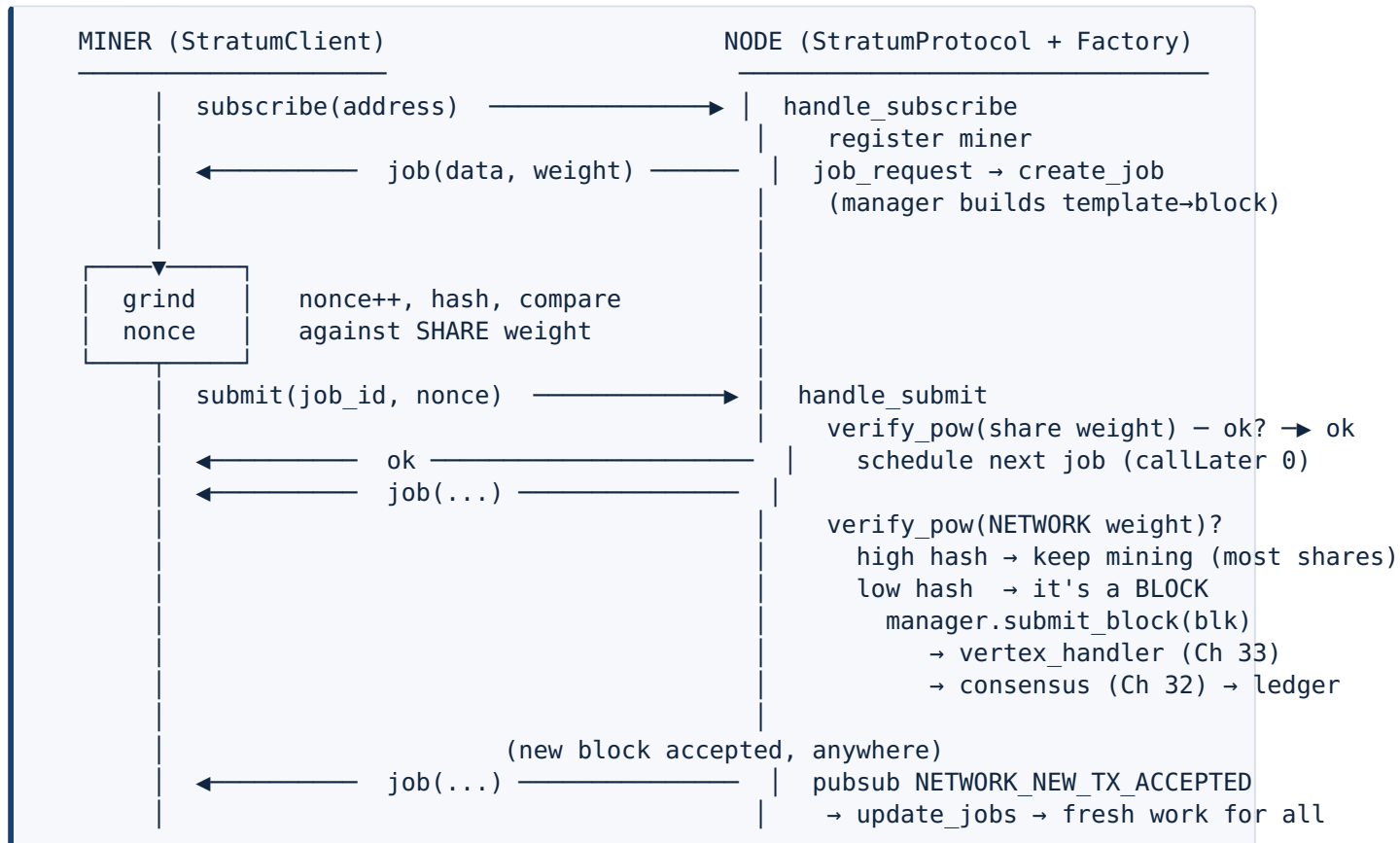
First it rejects a block whose parent is no longer the best tip — stale work that lost the race. Then it warns (but accepts) a block whose weight is below the significance floor. Finally it calls `propagate_tx` (`manager.py:851`), which routes into `on_new_tx` (`:864`) and from there to the vertex handler:

```
success = self.vertex_handler.on_new_relayed_vertex(vertex, ...)
if propagate_to_peers and success:
    self.connections.send_tx_to_peers(vertex)
```

`hathor/manager.py:878`

From here the block is just a vertex. `on_new_relayed_vertex` (Ch. 33) runs it through full verification (Ch. 31), consensus (Ch. 32), storage and indexing (Ch. 27–28), and finally re-broadcasts it to peers. The miner's block has become part of the ledger, and — through the pub-sub event in §37.4.4 — every other connected miner is immediately handed fresh work built on top of it. The loop closes.

37.4.7 The job/share lifecycle, end to end



37.5 Why Stratum, and not naive polling

The standalone `hathor-cli mining` tool (§37.4.3) shows the simple alternative:

getwork-style HTTP polling. The miner sends an HTTP request, gets a block back, mines it to completion, then sends another request. Why does production use Stratum instead?

Staleness. With polling there is no live link. If a new block lands on the network the instant after a miner fetched work, that miner grinds doomed, already-stale work until its next poll — wasted electricity. Stratum's factory subscribes to the node's "new block accepted" event and pushes fresh jobs the moment the tip moves (§37.4.4). Wasted work shrinks to the round-trip latency.

Granularity and hash-rate measurement. Polling only reveals a miner's effort when it finds a whole block — which, for a single machine against network difficulty, may be never. Stratum's share model asks the miner to submit partial solutions against an easy, per-miner difficulty, tuned so each miner reports roughly every 5 seconds (`AVERAGE_JOB_TIME`, `stratum.py:348`). The node continuously sees each miner is alive and how fast it hashes (`estimated_hash_rate`, `stratum.py:714`) — essential for pool accounting, and impossible with bare polling.

Connection cost. Polling opens a fresh TCP connection (and re-fetches state) every cycle. Stratum holds one long-lived connection per miner, amortizing setup and letting the node push rather than the miner pull.

A standard, not a bespoke API. Stratum is the de-facto protocol the whole mining ecosystem speaks (the docstring at `stratum.py:341` cites the Bitcoin Stratum references). By implementing it, Hathor lets existing mining software — written for Bitcoin and other chains — point at a Hathor node with minimal change. A bespoke HTTP scheme would force every miner vendor to write Hathor-specific code. Interoperability is the deciding trade-off: Hathor adopts the ecosystem's protocol rather than inventing its own.

The cost of Stratum is complexity — a stateful server, per-miner job history, share-difficulty tuning, timeouts. For a protocol that must serve real hardware competing for real money, that complexity pays for itself. For a one-off test block, the polling miner is perfectly adequate, which is why both exist.

37.6 How it plugs into the lifecycle

Mining is wired in at three points along the node's life (Ch. 29):

- **Build time.** If `--stratum` was passed, the builder constructs the `StratumFactory` and attaches it to the manager (`hathor_cli/builder.py:374` , and `hathor/builder/builder.py:449` for the test-side `Builder`). The `CpuMiningService` is always injected into the manager (`hathor/manager.py:112`), whether or not Stratum is enabled.
- **Listen time.** `run_node` opens the Stratum TCP port if `--stratum` is set, pointing the listener at the factory (`hathor_cli/run_node.py:226`).
- **Start time.** `HathorManager.start` (`manager.py:336`) calls `stratum_factory.start()` , which subscribes the factory to the pub-sub bus so new blocks trigger fresh jobs. On stop (`manager.py:371`), the factory is torn down.

CPU mining, by contrast, never runs inside the node's reactor. It runs in separate processes — the CLI tools' worker `Process` es (`hathor_cli/mining.py:125` , `hathor_cli/stratum_mining.py`) — keeping the long grind off the event loop, exactly as the Chapter 2 / Chapter 16 rule demands.

And every solved block, by whatever route, funnels through `submit_block` → `on_new_relayed_vertex` → the verification → consensus → storage pipeline of Chapters 31–33. Mining produces; the rest of the node ingests.

Recap

Concept	Where it lives	Central type / method
The candidate block (work)	<code>hathor/mining/block_template.py:29</code>	<code>BlockTemplate</code> (NamedTuple)
Filling a template into a block	<code>block_template.py:53</code>	<code>BlockTemplate.generate_mining_block</code>
Node builds the template	<code>hathor/manager.py:696</code> , <code>:666</code>	<code>_make_block_template</code> , <code>make_block_template</code>
Reward & weight decisions	<code>manager.py:734</code> , <code>:742</code>	DAA: <code>calculate_next_weight</code> , <code>get_tokens_issued_per_block</code>
Solving proof-of-work on a CPU	<code>hathor/mining/cpu_mining_service.py:50</code>	<code>CpuMiningService.start_mining</code>
Standalone polling CPU miner	<code>hathor_cli/mining.py:35</code>	<code>worker</code> → <code>CpuMiningService</code>
Stratum transport (JSON-RPC)	<code>hathor/stratum/stratum.py:181</code>	<code>JSONRPC</code> (LineReceiver)
Stratum server (per miner)	<code>stratum.py:337</code>	<code>StratumProtocol</code>
Subscribe → first job	<code>stratum.py:441</code> , <code>:619</code>	<code>handle_subscribe</code> , <code>create_job</code>
Share check → block detection	<code>stratum.py:479</code>	<code>handle_submit</code> (two <code>verify_pow</code> calls)
Per-miner share difficulty	<code>stratum.py:693</code>	<code>calculate_share_weight</code>
Push fresh work on new tip	<code>stratum.py:773</code> , <code>:765</code>	<code>StratumFactory.start</code> , <code>update_jobs</code>

Concept	Where it lives	Central type / method
Standalone Stratum CPU miner	<code>hathor/stratum/stratum.py:813</code>	<code>StratumClient</code>
Merged mining coordinator	<code>hathor/merged_mining/coordinator.py:1124</code>	<code>MergedMiningCoordinator</code>
Auxiliary proof-of-work	<code>hathor/transaction/aux_pow.py:24</code>	<code>BitcoinAuxPow</code>
Solved block back into the node	<code>hathor/manager.py:796</code>	<code>submit_block</code> → <code>propagate_tx</code>

Mining is the node's monetary edge: it produces the blocks that mint HTR and that consensus uses to order the DAG. The node's role is narrow and well-bounded — build a correct **template** (parents, reward, weight; §37.4.1–2), hand it out as Stratum **jobs**, accept **shares** as proof of effort, and recognize the rare share that is also a **block** (§37.4.4). The heavy nonce-grinding is deliberately kept outside the reactor, in separate processes or external hardware (§37.3, §37.6). Whatever produces a block — local CPU, Stratum hardware, or Bitcoin merged mining (§37.4.5) — every solution re-enters through `submit_block` and travels the same ingestion pipeline as any peer-relayed vertex (§37.4.6). The next chapter, **Chapter 38 — Feature Activation**, returns to the `signal_bits` field we saw the template carry: it is the channel through which miners vote to switch on protocol upgrades — including changes to the very mining and consensus rules this chapter rests on — without a disruptive flag day.

1. A nonce ("number used once") is a counter field in a block whose only purpose is to be varied during mining. Changing the nonce changes the block's hash; the miner tries nonce after nonce until the hash falls below the target. A Hathor block's nonce is 16 bytes; a transaction's is 4. ↩
2. A `NamedTuple` is an immutable Python record type: like a tuple, but with named fields (`template.reward` instead of `template[1]`). Once created it cannot be changed, which suits a template — it describes a fixed snapshot of work. ↩
3. **DAA** = Difficulty Adjustment Algorithm. The rule that raises or lowers required block weight so blocks keep arriving at a steady target rate as total network hash power changes. Full treatment in Chapter 9. ↩
4. JSON-RPC 2.0 is a tiny standard for remote procedure calls encoded as JSON: each message is an object with a `method`, `params`, and an `id` (for requests), or a `result / error` (for responses). Stratum is JSON-RPC sent one message per line over a raw TCP connection. ↩

5. In Twisted, a factory is the object that creates a fresh protocol instance for each incoming connection and holds state shared across all of them. The `StratumFactory` builds one `StratumProtocol` per miner and holds the list of all miners. Full treatment of factories in Chapter 16 (and the pattern in Chapter 3). ↩
6. A Merkle path (or Merkle branch) is the short list of sibling hashes that lets you prove a single item belongs to a set summarized by one root hash, without revealing the whole set. Merged mining uses it to prove the Hathor block's hash was committed inside Bitcoin's coinbase transaction. ↩